DCF Valuation & Financial Analytics Platform

Institutional Equity Valuation • Reverse DCF (BrentQ) • 3-Stage DuPont • Multi-Currency Ingestion
GitHub Repository: https://github.com/ivahcomworkspace-droid/dcf-valuation-model • Production v1.0.0

DCF Valuation Engine | Reverse DCF (BrentQ) | 3-Stage DuPont ROE | Live yfinance Ingestion | Multi-Currency (₹, $, €, £, ¥) | Streamlit Cloud Ready

1. Executive Summary & Vision

The **DCF Valuation & Financial Analytics Platform** is an institutional-grade financial application designed for equity research analysts, quantitative investors, and corporate finance practitioners. It unifies classical Discounted Cash Flow valuation, market-implied growth solving (Reverse DCF), and DuPont operational decomposition into an interactive visual dashboard.

Built on clean architectural principles, the system decouples a network-isolated financial math engine from an external market ingestion layer and a reactive Streamlit/Plotly frontend. It features real-time multi-currency support, in-memory TTL caching, resilient error boundaries, and a 14-test deterministic unit suite.

INTRINSIC VALUE
$92.12
Per Share (Benchmark Baseline)

ENTERPRISE VALUE
$1,992M
75.6% Terminal Value Contribution

EQUITY VALUE
$1,842M
EV + $50M Cash - $200M Debt

2. System Architecture & Modular Organization

Layer / Module	Key Files	Core Responsibilities
Domain Engine `fin_engine/`	`dcf.py` `reverse_dcf.py` `dupont.py`	**Zero network dependencies.** Pure financial domain math: 5-year compounding cash flows, Gordon Growth terminal value, 2D sensitivity matrices, Brent numerical root solver, and DuPont exact-path delta attribution.
Ingestion & Services `services/`	`schemas.py` `ingestion.py`	Pydantic validation models for market snapshots and balance sheet line items. Live data fetching via `yfinance` with a 5-minute in-memory TTL cache and dynamic currency mapping.
Presentation Layer `dashboard/`	`app.py` `.streamlit/config.toml`	Production Streamlit web dashboard. Features dark obsidian/slate Plotly dual-bar cash flow projections, 2D sensitivity heatmaps, reactive parameter sliders, and preset scenarios.
Verification Suite `tests/`	`test_dcf.py` , `test_reverse_dcf.py` `test_dupont.py` , `test_services.py`	Automated test harness with 14 unit tests executed via `pytest` , verifying valuation accuracy down to the exact cent, numerical solver parity, and currency symbol resolution.

3. Discounted Cash Flow (DCF) Mathematical Engine

The valuation engine executes a 5-year explicit discrete forecast followed by perpetual terminal value capitalization under the Gordon Growth Model.

3.1. 5-Year Explicit Forecast Schedule

```
FCFF_t = FCFF_0 × (1 + g_1)^t
PV(FCFF_t) = FCFF_t / (1 + WACC)^t
```

Year	Compounding Formula	Projected FCFF	Discount Factor	Present Value (PV)
Year 1	$100.00M × (1 + 0.08)^1	**$108.00M**	1 / 1.09^1 = 0.9174	**$99.08M**
Year 2	$100.00M × (1 + 0.08)^2	**$116.64M**	1 / 1.09^2 = 0.8417	**$98.17M**
Year 3	$100.00M × (1 + 0.08)^3	**$125.97M**	1 / 1.09^3 = 0.7722	**$97.27M**
Year 4	$100.00M × (1 + 0.08)^4	**$136.05M**	1 / 1.09^4 = 0.7084	**$96.38M**
Year 5	$100.00M × (1 + 0.08)^5	**$146.93M**	1 / 1.09^5 = 0.6499	**$95.50M**
			Sum of Explicit Present Values (Σ PV_FCFF):	**$486.41M**

3.2. Gordon Growth Terminal Value & Enterprise Value Bridge

```
FCFF_6 = FCFF_5 × (1 + g_2) = $146.9328M × 1.025 = $150.606M
Terminal Value_5 = FCFF_6 / (WACC - g_2) = $150.606M / (0.09 - 0.025) = $2,317.02M
PV(Terminal Value) = Terminal Value_5 / (1 + WACC)^5 = $2,317.02M / 1.09^5 = $1,505.90M
Enterprise Value = Σ PV(FCFF) + PV(Terminal Value) = $486.41M + $1,505.90M = $1,992.31M (~$1,992M)
Terminal Value % of EV = $1,505.90M / $1,992.31M = 75.59% (~76%)
Equity Value = Enterprise Value + Cash - Debt = $1,992.31M + $50.0M - $200.0M = $1,842.31M (~$1,842M)
Intrinsic Value Per Share = Equity Value / Diluted Shares = $1,842.31M / 20.0M = $92.12 per share
```

3.3. 2D Sensitivity Matrix: Share Price ($)

Displays share price sensitivity across WACC ($7.0\% - 11.0\%$) and Long-Term Growth ($1.5\% - 3.5\%$):

WACC \ LTG	1.5%	2.0%	2.5% (Baseline)	3.0%	3.5%
7.0%	$114.9	$125.1	$137.5	$153.1	$173.1
8.0%	$95.6	$102.5	$110.7	$120.5	$132.5
9.0% (Baseline)	$81.4	$86.4	**$92.1 ☆**	$98.8	$106.7
10.0%	$70.6	$74.3	$78.5	$83.3	$88.8
11.0%	$62.1	$65.0	$68.1	$71.7	$75.7

4. Reverse DCF: Solving Market-Implied Growth Rates

Reverse DCF inverts the valuation equation: given the observable market stock price P_{market}, it solves for the 5-year annual growth rate (g_1^*) implied by the market.

```
Target Enterprise Value = P_market × Shares + Total Debt - Cash
f(g_1) = IntrinsicValue(g_1) - P_market = 0
```

- **Primary Solver (`scipy.optimize.brentq`)**: Brent's hybrid root-finding algorithm converges to 10^{-6} tolerance in < 10 iterations.
- **Binary Search Fallback**: Custom bisection solver with dynamic bracket expansion guarantees execution across all environments.
- **Verification**: Evaluates forward DCF on g_1^* to prove convergence with `|validation_price - target_price| < 1e-4`.

Market Share Price	Implied 5-Yr Growth Rate	Target Enterprise Value	Market Interpretation
$60.00 (Discount)	**-1.05%**	$1,350.0M	Priced for cash flow contraction
$75.00	**+3.55%**	$1,650.0M	Priced for conservative GDP-like growth
$92.12 (Baseline)	**+8.00%**	$1,992.4M	Exact benchmark parity
$110.00 (Premium)	**+12.01%**	$2,350.0M	Priced for double-digit growth
$140.00 (High Growth)	**+17.71%**	$2,950.0M	Requires aggressive market expansion

5. 3-Stage DuPont ROE Decomposition & Delta Attribution

Deconstructs Return on Equity (ROE) into operational profitability, asset turnover, and financial leverage:

```
ROE = (Net Income / Revenue) × (Revenue / Total Assets) × (Total Assets / Equity)
ROE = Net Profit Margin × Asset Turnover × Equity Multiplier
```

5.1. Exact-Path Sequential Delta Attribution

When comparing two periods ($T_0 \to T_1$), sequential path substitution attributes changes with **0% residual**:

```
Δ ROE = Margin Effect + Turnover Effect + Leverage Effect
• Margin Effect = (Margin_1 - Margin_0) × Turnover_0 × Leverage_0
• Turnover Effect = Margin_1 × (Turnover_1 - Turnover_0) × Leverage_0
• Leverage Effect = Margin_1 × Turnover_1 × (Leverage_1 - Leverage_0)
```

Benchmark Case Study:
Period T_0: Margin = 10.0%, Turnover = 0.80x, Leverage = 2.50x → **ROE = 20.0%**
Period T_1: Margin = 12.0%, Turnover = 0.83x, Leverage = 2.40x → **ROE = 24.0%** (Δ ROE = +4.0%)
• Margin Expansion: **+4.00%** (100% of gain) • Asset Turnover: **+1.00%** (+25% of gain) • Deleveraging: **-1.00%** (-25% drag).

6. Live Ingestion, Multi-Currency & Error Boundaries

- **Multi-Currency Auto-Detection**: Ingested tickers automatically format the entire dashboard in reporting currency (e.g. `₹` for `TATASTEEL.NS`, `$` for `AAPL`, `€` for `MC.PA`, `£` for `AZN.L`, `¥` for `7203.T`). Includes a manual currency override selector.
- **In-Memory TTL Caching**: Thread-safe cache with a 300-second TTL prevents repeated API calls and eliminates rate-limiting.
- **Graceful Error Boundaries**: Empty-input guards, custom `TickerNotFoundError` handlers with international exchange suffix guidance, and balance sheet disclosure disclosures ensure the app never crashes.

7. Production Deployment & Cloud Infrastructure

Component	Specification & Implementation
Theme Configuration	`.streamlit/config.toml`: Obsidian dark palette (`#0b0f17` background, `#3b82f6` accent), headless server mode, telemetry disabled.
Dependencies	`requirements.txt`: Constrained versions of `streamlit`, `plotly`, `yfinance`, `scipy`, `pandas`, `numpy`, `pydantic`, and `pytest`.
Streamlit Cloud Deploy	Deploy from GitHub repo `ivahcomworkspace-droid/dcf-valuation-model` → Main file path: `dashboard/app.py`.
Quality Assurance	`python -m pytest tests/ -v`: 14 passing unit tests covering DCF math, reverse DCF parity, DuPont attribution, and currency resolution.